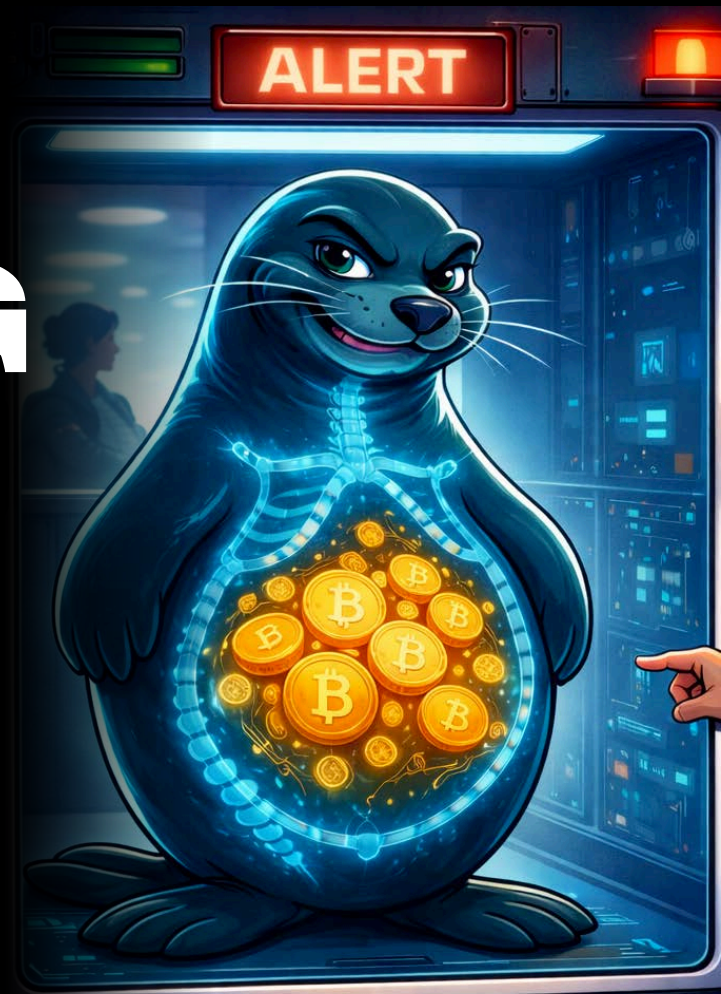


COMPILED V8 JAVASCRIPT BYTECODE MALWARE

BREAKING THE SEAL

Static deobfuscation of compiled
V8 JavaScript bytecode
malware

Aleksandra “Hasherezade” Doniec · Check Point .JSC → READABLE
Research CODE



WHOAMI

ABOUT ME

Aleksandra “Hasherezade” Doniec [@hasherezade](#)

- Malware researcher & engineer at [Check Point Research](#)
- I reverse malware & build open-source tools
- Author of [PE-sieve/HollowsHunter](#) · [Tiny Tracer](#) · [PE-bear](#)
- Today: deobfuscating a [V8 cache payload](#) & publishing the toolkit



[hasherezade.net](#)



[@hasherezade](#)
tools · write-ups

WHEN THE TOOLS FIT

Analysis is easier when the payload arrives in a form our tools already understand.

01

A **readable script**

02

An **intermediate language** we can decompile

03

A **native binary** for IDA & mature frameworks

The tools aren't magic — but they give us a place to start.

INTRODUCTION

THE USUAL OBSTACLES

Malware authors do anything they can to **disrupt our workflow**.

THE USUAL OBSTACLES

EVASION OBFUSCATION

Before	After
<pre>__int64 sub_4010B8() { printf("Hello, world!"); return 0i64; }</pre>	<pre>__int64 sub_401000() { *(_DWORD *)(a2 + v2) += v2; if (!(_DWORD)a2) JUMPQUT(0x401024i64); MEMORY[0x100C3C9] &= a2; return (loc_4017B1)(asc_404159); }</pre>

Example via github.com/DosX-dev/obfus.h

INTRODUCTION

FRICTION BY FORMAT

A quieter strategy: choose a format where the ecosystem is weak.

If the payload lands where our tools don't quite fit, the attacker gets friction almost for free.

WE'VE SEEN THIS BEFORE

GO RUST AOT .NET

Each created its own reversing friction. The tools caught up — slowly.

```
0 int64_t* std::thread::spawn::h062b21a91c7551cf(int64_t* arg1)

std::thread::spawn::h062b21a91c7551cf:
004021c0 sub     rsp, 0xa8
004021c7 mov     qword [rsp+0x30 {var_78}], rcx
004021cc mov     qword [rsp+0x38 {var_70}], rcx
004021d1 mov     rax, 0x8000000000000000
004021db mov     qword [rsp+0x90 {var_18}], rax {-0x8000000000000000}
004021e3 mov     rax, qword [rsp+0x90 {var_18}] {-0x8000000000000000}
004021eb mov     qword [rsp+0x70 {var_38}], rax {-0x8000000000000000}
004021f0 mov     rax, qword [rsp+0x98 {var_10}]
004021f8 mov     qword [rsp+0x78 {var_30}], rax
004021fd mov     rax, qword [rsp+0xa0 {var_8}]
00402205 mov     qword [rsp+0x80 {var_28}], rax
0040220d mov     rcx, qword [rel _rdata] {0x0}
00402214 mov     rax, qword [rel data_4f50e8] {0x0}
0040221b mov     qword [rsp+0x60 {var_48}], rcx {0x0}
00402220 mov     qword [rsp+0x68 {var_40}], rax {0x0}
00402225 mov     byte [rsp+0x88 {var_20}], 0x0
0040222d lea     rcx, [rsp+0x40 {var_68}]
00402232 lea     rdx, [rsp+0x60 {var_48}]
00402237 call   std::thread::Builder::spawn_unchecked::hfddd97d24214fbac
0040223c mov     rcx, qword [rsp+0x30 {var_78}]
00402241 lea     rdx, [rsp+0x40 {var_68}]
00402246 lea     r8, [rel data_4f50f0] {"failed to spawn thread/rustc/4eb161250e340c8f48f66e2b929ef4a5be"}
0040224d mov     r9d, 0x16
00402253 lea     rax, [rel data_4f5158]
0040225a mov     qword [rsp+0x20 {var_80}], rax {data_4f5158}
0040225f call   core::result::Result<T,E>::expect::h1f41d992ae554c1f
00402264 mov     rax, qword [rsp+0x38 {var_70}]
00402269 add     rsp, 0xa8
```

Rust binary disassembly · via cxiao.net/posts/2025-08-17-not-so-simple-rust-loader

ANOTHER SOURCE OF FRICTION

COMPILED V8 BYTECODE

WHY THIS FORMAT MATTERS

WHY JAVASCRIPT, WHY NODE

```
<!DOCTYPE html>
<html>
<body>
<h1>My First JavaScript</h1>

<button type="button"
onclick="document.getElementById('demo').innerHTML =
Date()">
Click me to display Date and Time.</button>

<p id="demo"></p>

</body>
</html>
```

My First JavaScript

Click me to display Date and Time.

Wed Jul 15 2026 21:40:39 GMT+0200 (Central European Summer Time)

Born to enhance web pages

browser language → a general-purpose platform via [Node.js](#)

[npm](#) ≈ [pip](#) — pull in existing modules.

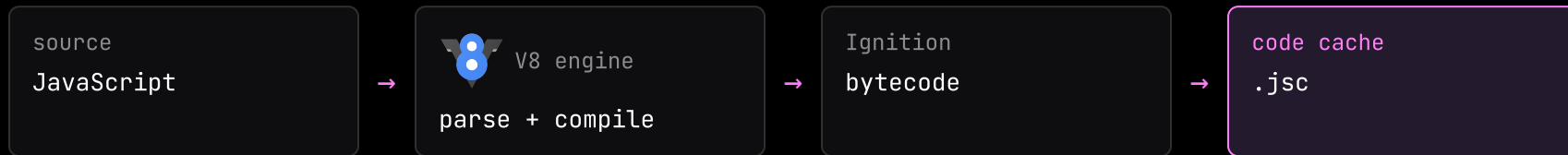
Build capable software quickly, in a high-level language, from parts that already exist.



```
$ npm install <module>
```

WHY THIS FORMAT MATTERS

THE COMPILATION TRICK



Bytecode is not a stable public format — V8-internal & version-specific. TurboFan later JITs hot paths to native, but the cache stores Ignition bytecode — and that's what ships to us.

WHY THIS FORMAT MATTERS

CODE CACHING IN PRACTICE

COMPILE → CACHE · MAKE_CACHE.JS

```
const vm = require("vm");
const fs = require("fs");
const source = fs.readFileSync("hello.js");
// compile, then serialize the bytecode
const script = new vm.Script(source);
const cache = script.createCachedData();
fs.writeFileSync("hello.js.cache", cache);
```

LOAD CACHE → RUN · RUN_CACHE.JS

```
const vm = require("vm");
const fs = require("fs");
const source = fs.readFileSync("hello.js",
"utf8");
const cache =
fs.readFileSync("hello.js.cache");
// reuse the cache – skip parse + compile
const script = new vm.Script(source, {
  cachedData: cache
});
if (script.cachedDataRejected)
  throw new Error("V8 rejected the cache");
script.runInThisContext();
```

WHY THIS FORMAT MATTERS

THE ATTACKER'S RECIPE

write JS

→

obfuscate

→

compile → V8 cache

→

bundle runtime

→

ship compiled

WHAT THEY KEEP

The productivity of JavaScript + npm — build fast, from parts that already exist.

WHAT THEY GAIN

A new layer of defense — as a bonus, for free.

THE ANALYST RECEIVES ONLY COMPILED V8 BYTECODE

THE CASE STUDY

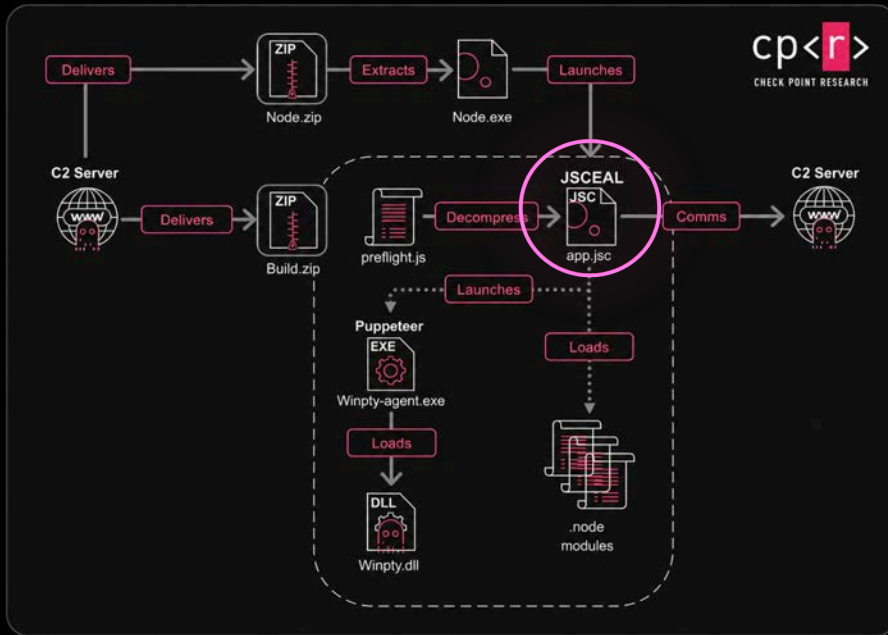
MEET JSCEAL

Named for the final payload's extension — `.jsc`

A cryptocurrency-focused stealer with surveillance & interception

Tracked by Check Point since early 2025 — campaigns already published

Today: the untold part — how we reverse-engineered it



downloading of the final payload

research.checkpoint.com/2025/jsceal-targets-crypto-apps

THE CASE STUDY

THE STARTING POINT — NOT FRIENDLY

Brotli-compressed blob

V8 code cache — run by a
bundled Node.js

obfuscated JavaScript (before
compilation)

THE USUAL GOAL

Recover code well enough to understand what it does

Compare samples

Track how the family evolves

THE QUESTION

HOW DO YOU ANALYZE IT WHEN THE SOURCE IS GONE?

browser + crypto theft

Telegram sessions

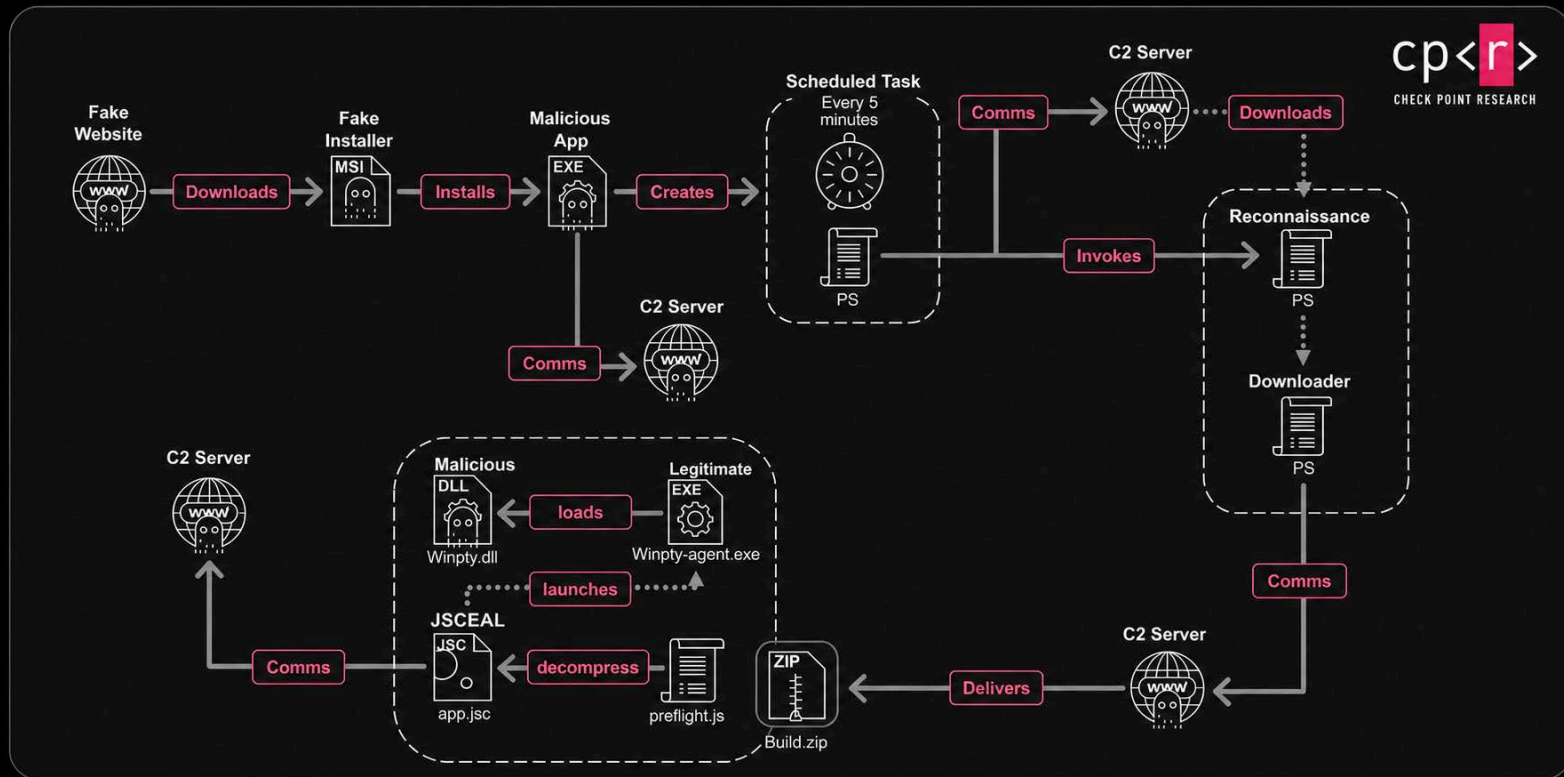
keylogging

screenshots

local HTTPS MITM proxy

FIRST ATTEMPT

RUN IT AND WATCH



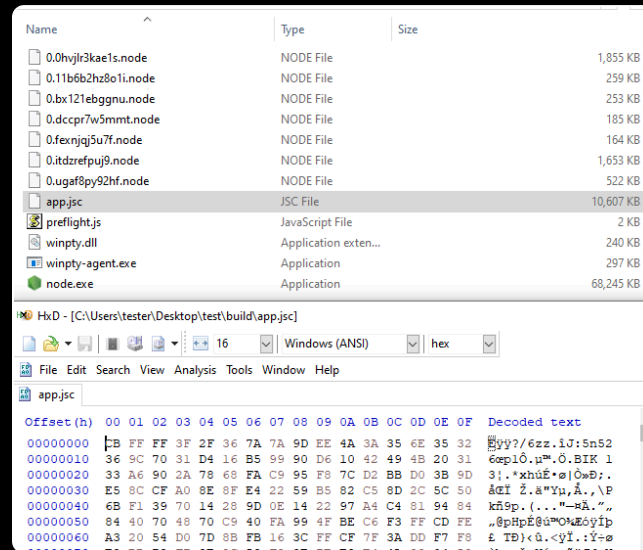
FIRST ATTEMPT

THE BUNDLE UNPACKED

I recreated what the deployment script does — download, unpack, run.

RUN IT THE WAY THE LOADER WOULD

```
# bring the payload's own runtime, load preflight first
.\node.exe -r .\preflight.js .\app.js
```



the bundle unpacked — `app.js` ·
`preflight.js` · `node.exe` · `.node`
modules

RUN IT • WHAT COMES OUT

THE SERVER WAS GONE

Running it exposes only the **surface**.

real run

```
Windows PowerShell - jsceal_last\build

PS C:\Users\tester\Desktop\jsceal_last\build> $RuntimeLastExecutionTime = Measure-Command {
>> $env:TOKEN = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3YXQ1OjE3MzMyNDIxNjcsInN1YiI6InZlcnNpb24ifQ.70Y1PGYqj65r0AyDYcq4iqJMSWq8BmtPY8FNHRnubow"
>> $env:TASK_NAME = $TaskName
>>
>> .\node.exe -n .\preflight.js .\app.js
>> }
Faro
Swamp/faro:transport-fetch
Failed sending payload to the receiver
{
  meta: {
    sdk: { version: '1.0.0' },
    app: { name: 'client' },
    user: { id: '01032c87-1d27-4314-a150-58f9774a975ec' },
    session: { id: 'VxjNQM812' }
  },
  events: [
    {
      name: 'session_start',
      domain: 'node',
      attributes: {},
      timestamp: '2020-08-09T00:29:59.180Z'
    }
  ]
}
}
[FetchError]: request to https://faro.vertical-scaling.com/collect failed, reason: getaddrinfo ENOTFOUND faro.vertical-scaling.com
at ClientRequest.<anonymous> (evalmachine.<anonymous>:1:12294373)
at ClientRequest.emit (node:events:513:28)
at TLSSocket.socketErrorListener (node:_http_client:502:9)
at TLSSocket.emit (node:events:513:28)
at emitErrorNT (node:internal/streams/destroy:151:8)
at emitErrorCloseNT (node:internal/streams/destroy:116:3)
at process.processTicksAndRejections (node:internal/process/task_queues:82:21) {
  type: 'system',
  errno: 'ENOTFOUND',
  code: 'ENOTFOUND'
}
```

The one thing it sends

```
events: [ {
  name: 'session_start',
  domain: 'node'
} ]
```

A single Faro telemetry beacon — one **session_start** event, nothing more.

Then it dies here

```
reason: getaddrinfo ENOTFOUND faro.vertical-scaling.com
```

The endpoint is already **gone** — **ENOTFOUND**. So we tried to bring it back ourselves.

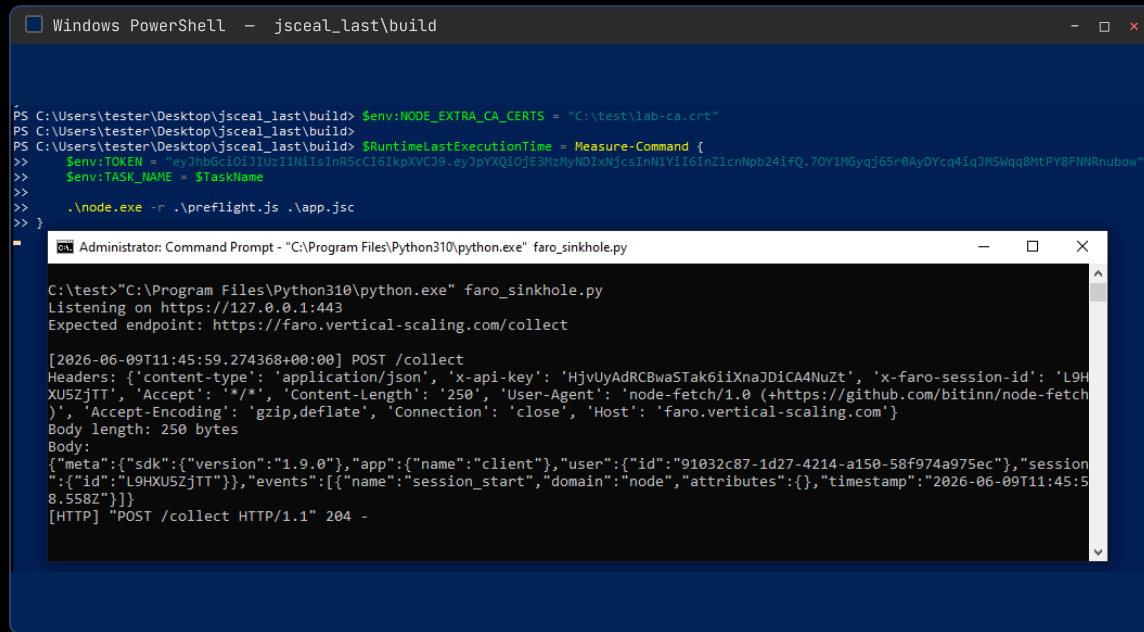
WE BROUGHT THE ENDPOINT BACK OURSELVES

WE BROUGHT IT BACK

Even a live server doesn't expose the **logic**.

sinkhole receives the only request · answers 204

- 1 Redirect the host — **hosts** → 127.0.0.1
- 2 Forge trust — self-signed cert via **NODE_EXTRA_CA_CERTS**
- 3 Serve — a small HTTPS sinkhole on **:443**



```
Windows PowerShell - jsceal_last\build

PS C:\Users\tester\Desktop\jsceal_last\build> $env:NODE_EXTRA_CA_CERTS = "C:\test\lab-ca.crt"
PS C:\Users\tester\Desktop\jsceal_last\build>
PS C:\Users\tester\Desktop\jsceal_last\build> $RuntimeLastExecutionTime = Measure-Command {
>>   $env:TOKEN = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpYXQiOiJlZ3RzPyNDIWMjc1NHk1Y1I6InZlcnNpb241fQ.70Y1MGYqj65r0AyDYcq41qJMSWqq8MtPY8FNnrubow"
>>   $env:TASK_NAME = $TaskName
>> }
>> .\node.exe -r .\preflight.js .\app.jsc
>> }
```

```
Administrator: Command Prompt - "C:\Program Files\Python310\python.exe" faro_sinkhole.py

C:\test>"C:\Program Files\Python310\python.exe" faro_sinkhole.py
Listening on https://127.0.0.1:443
Expected endpoint: https://faro.vertical-scaling.com/collect

[2026-06-09T11:45:59.274368+00:00] POST /collect
Headers: {'content-type': 'application/json', 'x-api-key': 'HjvUyAdRCBwaSTak6iiXnaJdICA4NuZt', 'x-faro-session-id': 'L9HXU5ZjTT', 'Accept': '*/', 'Content-Length': '250', 'User-Agent': 'node-fetch/1.0 (+https://github.com/bitinn/node-fetch)', 'Accept-Encoding': 'gzip,deflate', 'Connection': 'close', 'Host': 'faro.vertical-scaling.com'}
Body length: 250 bytes
Body:
{"meta":{"sdk":{"version":"1.9.0"},"app":{"name":"client"},"user":{"id":"91032c87-1d27-4214-a150-58f974a975ec"},"session":{"id":"L9HXU5ZjTT"}}, "events":[{"name":"session_start","domain":"node","attributes":{},"timestamp":"2026-06-09T11:45:58.558Z"}]}
[HTTP] "POST /collect HTTP/1.1" 204 -
```

It connects with status **204** — yet still exposes only the same **session_start** beacon.

FIRST ATTEMPT

BLACK BOX ANALYSIS IS NOT ENOUGH

DYNAMIC GIVES US

Observed behavior — from one execution.

WE NEEDED

A repeatable way to recover full logic

To compare samples

To map the complete capability set



this path is not for the faint-
hearted

So I decided to **recover the program from the bytecode.**

GETTING TO CLEAN BYTECODE

TOOLING FOR .JSC IS THIN

decompiler

JSC Decompiler

online · needs upload

commercial · per .jsc file

Standard — up to 500 KB · \$200

Large — up to 2 MB · \$500

decompiler

ghidra_nodejs

OSS plugin · unmaintained

decompiler · chosen

View8

OSS Python · author at CPR

LLM on raw disassembly? Up to ~500 MB per payload — and the wrong abstraction level. **Abandoned.***

* A decision made at a particular moment in time — LLM context windows were still too short to process files this large.

GETTING TO CLEAN BYTECODE

WHY VIEW8

Open-source **Python** decompiler

By Moshe Marelus, Check Point Research

Already used on compiled V8 JavaScript malware

Didn't solve the whole problem — but a strong foundation I could extend

VIEW8

.jsc disassembly →
readable pseudocode

github.com/suleram/View8

[research.checkpoint.com – release
announcement](https://research.checkpoint.com/release-announcement)

THE NAIVE PATH FAILS

- ✗ Node treats the cache blob as a script — syntax error

[illegible]

GETTING THE DISASSEMBLY

Skip Node entirely — build a disassembler **on the V8 API** itself.

- build the matching version of V8
- apply the author's patches, link as a static library
- consume the code cache directly

repo github.com/j4k0xb/View8

THE FIX — A DISASSEMBLER ON THE V8 API

```
V8::SetFlagsFromString("--no-lazy --no-flush-bytecode");
V8::Initialize();
isolate = Isolate::New(create_params);

auto* cached = new ScriptCompiler::CachedData(buf, len);
ScriptOrigin origin = CreateScriptOrigin(
    String::NewFromUtf8Literal(isolate, "code.jsc"));
ScriptCompiler::Source source(dummySource, origin, cached);

ScriptCompiler::CompileUnboundScript(
    isolate, &source, ScriptCompiler::kConsumeCodeCache);
```

GETTING TO CLEAN BYTECODE

DEBUGGING V8 ITSELF

A string-printing bug in the V8 disassembly path

It passed a 16-bit code unit to `std::isprint()`, then narrowed printable values to `char` — undefined for non-byte inputs, and capable of garbling wide characters

Result: raw unprintable bytes injected mid-string — corrupt disassembly that broke View8 downstream

No workaround — I debugged V8 and fixed it at the root cause

Released: a ready-made disassembler you can reuse

THE ROOT-CAUSE PATCH

```
accumulator→Add("\\n");
} else if (c == '\\') {
    accumulator→Add("\\\\");
- } else if (!std::isprint(c)) {
+ } else if (c ≥ 0x20 && c ≤ 0x7e) {
+     accumulator→Put(static_cast<char>(c));
+ } else if (c ≤ 0xff) {
    accumulator→Add("\\x%02x", c);
    } else {
-     accumulator→Put(static_cast<char>(c));
+     accumulator→Add("\\u%04x", c);
    }
```

Details: github.com/hasherezade/jsc_deobfuscator/wiki/Building-V8-Disasm

READING THE OUTPUT

THE FALSE SUMMIT

DECOMPILED OUTPUT · WHAT YOU ACTUALLY OPEN

**34—
55** MB

cache

**330—
500** MB

disassembly

~50 MB

decompiled

- Too big to read linearly — tens of thousands of functions, most of it bundled dependency code.
- The JavaScript was obfuscated before it was compiled — the malware logic stayed hidden.

A milestone — not yet the summit.

```
// ... thousands of functions like this ...
function func_Mz_0x10000000f(a0)
{
    r5 = Scope[0]
    r2 = func_r_0x10000000b
    r0 = new {"w": 126853, "d": "741^", "J": 166834, "m": "!Rgf", "H": 187083, "U": "*La6", "F": 1182, "r": "Zj4P", "I": 37217, "x": "
Scope[6705][2] = new {"w": 1342}
    r6 = new {"j6B6z": null, "hBPBb": null, "qby0P": null, "ykkYm": null, "SeAyf": null, "yHrsY": null, "umIdy": null, "RBgqe": null}
    r7 = func_r_0x10000000b(r0["w"], r0["d"])
    r6["j6B6z"] = (r7 + func_r_0x10000000b(r0["J"], r0["m"]))
    r6["hBPBb"] = func_hBPBb_0x10000000c
    r6["qby0P"] = func_r_0x10000000b(r0["H"], r0["U"])
    r6["ykkYm"] = func_ykkYm_0x10000000d
    r6["SeAyf"] = func_SeAyf_0x10000000e
    r6["yHrsY"] = func_r_0x10000000b(r0["F"], r0["r"])
    r6["umIdy"] = func_r_0x10000000b(r0["I"], r0["x"])
    r7 = func_r_0x10000000b(r0["X"], r0["p"])
    r7 = (r7 + func_r_0x10000000b(r0["a"], r0["g"]))
    r7 = (r7 + func_r_0x10000000b(r0["h"], r0["p"]))
    r6["RBgqe"] = (r7 + "l")
    r1 = r6
    r7 = r1[func_r_0x10000000b(r0["k"], r0["z"])]
    r6 = r7[func_r_0x10000000b(r0["b"], r0["c"])]
    r3 = r6["l"]
    r4 = 0
    while (true)
    {
        r7 = Number(r4)
        r4 = (Number(r4) + 1)
        r6 = r3[r7]
        if (!r6 === "0")
        {
            if (!r6 === "1")
            {
                if (!r6 === "2")
                {

```


COMPILED CACHE → PSEUDOCODE

VIEW8 - A WORKED EXAMPLE

A V8 cache is a compiled artifact — names & formatting are already gone. What View8 recovers is lower-level: constants, registers, scopes, function boundaries, control flow.

ORIGINAL SOURCE

```
function greet(name) {  
  return "Hello " + name;  
}  
console.log(greet("BlackHat"));
```



VIEW8 RECONSTRUCTION • NAMES SHORTENED FOR DISPLAY

```
function func_start() {  
  ACCU = DeclareGlobals([func_greet, 0], <closure>)  
  r3 = func_greet("BlackHat")  
  r0 = console["log"](r3)  
  return r0  
}  
function func_greet(a0) {  
  return ("Hello " + a0)  
}
```

Not original JavaScript. Not directly runnable. But parseable and understandable. First milestone — a reliable path from `.jsc` to pseudocode.

WHAT VIEW8 GIVES US

REGISTERS, SCOPES, DECLARERS

Not named locals — virtual registers `r0` `r1` `r2` + accumulator `ACCU`

Functions declare other functions & pass data through `Scope`

DECOMPILED INTERNALS · FUNC_START

```
# func_start_0x1d1b0e89dcd9
# Declarer: None
# Const Pool
[func_greet_0x1d1b0e89ddc9, 0], "console", "log",
"greet", "BlackHat"
# Arguments: 1   # Registers: 5
# Code
{
  ACCU = DeclareGlobals([func_greet_0x1d1b0e89ddc9,
0], <closure>)
  r3 = func_greet_0x1d1b0e89ddc9("BlackHat")
  r0 = console["log"](r3)
  return r0
}
```

DECOMPILED INTERNALS · FUNC_GREET

```
# func_greet_0x1d1b0e89ddc9
# Declarer: func_start_0x1d1b0e89dcd9
# Const Pool
"Hello "
# Arguments: 2   # Registers: 1
# Code
{
  return ("Hello " + a0)
}
```

TRACE IT UP THE DECLARER TREE

SCOPE RESOLUTION

- Some values are propagated through **Scope**.
- They can be **set** by functions **higher** in the declarer hierarchy and **referenced** by functions **below** them.

declarer function

```
Scope[10083][2] = new { "c": 742 }
```

↓ referenced below by the child function

child function

```
r1 = (a1 - (-Scope[10083][2]["c"]))
```

↓ resolves to

```
r1 = (a1 - (-742))
```

IDENTIFYING THE OBFUSCATION

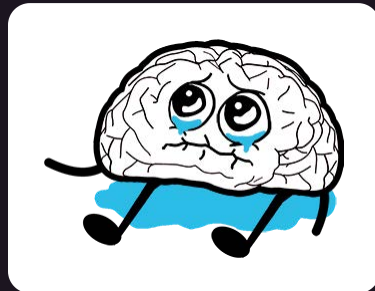
KNOW YOUR OBFUSCATOR

OBFUSCATED SOURCE • PREFLIGHT.JS

```
// fragment – one of the preflight.js scripts
// (may or may not share the same obfuscator)
var _0x3f=['c2NyaXB0','c3Jj','YXBwZW5k','aGVhZA=='];
var s=_0x1d(0x0); // _0x1d decodes → "script"
var e=d[_0x1d(0x4)](s); // → createElement("script")
var _0x2a='2|0|3|1'; // block-order string
while (!![]){
switch(_0x2a[_0x1d(0x7)]()){
case '0':
e[_0x1d(0x1)]=u; // → e.src = u
continue;
case '1':
h[_0x1d(0x2)](e); // → head.append(e)
continue;
// cases '2','3' ...
}
break;
}
```

Every real string is fetched through `_0x1d(0xN)` from an encoded array; the block order hides behind a `while / switch` dispatcher.

THE OBFUSCATOR



javascript-
obfuscator

open-source • documented • configurable

[github.com/javascript-obfuscator/javascript-o
bfuscator](https://github.com/javascript-obfuscator/javascript-obfuscator)

FOUR LAYERS, STACKED

1 String obfuscation

The important strings are hidden.

2 Control-flow flattening

Functions become state machines behind a dispatcher.

3 Proxy indirection

Chained forwarding layers, not the real call.

4 Operation wrappers

Even add / subtract / compare become functions.

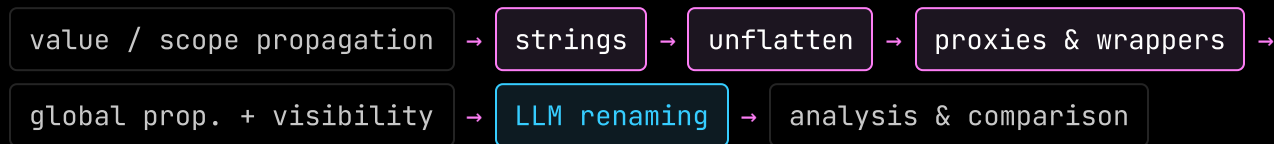
None surprising alone. The hard part: **they're stacked and depend on each other.**

 **DISCLAIMER** For the sake of brevity, some details here are omitted or simplified. A more comprehensive description will be available soon on our blog.

THE ARCHITECTURE

THE DEOBFUSCATION PIPELINE

DEOBFUSCATE — ORDER DICTATED BY DEPENDENCIES

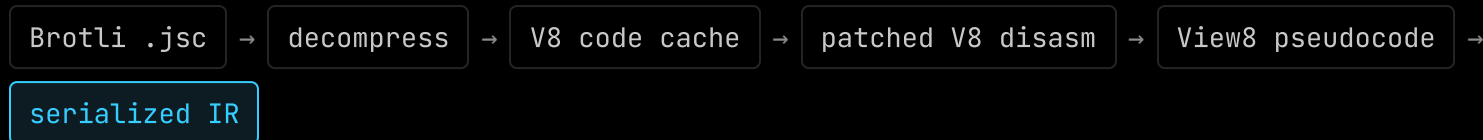


Strings must come first — every later layer depends on them. We'll follow [one small example function](#) through each pass.

THE ARCHITECTURE

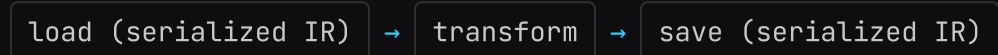
CHANGES IN VIEW8

DECOMPILE



NEW OUTPUT (AND INPUT): SERIALIZED IR

EACH FILTER CONSUMES THE PICKLE



Filters see the decompiler's real objects & function hierarchy

The deobfuscator stays a [separate tool](#) — not a tightly coupled fork.

THE ARCHITECTURE

CHANGES IN VIEW8

HIDING FUNCTIONS

After a pass removes a layer, it hides its helpers — once strings are recovered, the thousands of string decoders disappear from the output.

ADDING METADATA

Annotate every function and every line with custom metadata — e.g. which obfuscation types were found — which makes later processing much easier.

OUTPUT SPLITTING

Instead of one huge file, print a selected function subtree: follow **declared-by**, **called-by** and **referenced-by** from a chosen root, to a chosen depth.

...plus more changes and bug fixes — all in the [repository changelog](#).

STRINGS COME FIRST

They unlock every later filter
– APIs, paths, keys, commands,
URLs.

straight from View8 • before any pass

```
r5 = func_n_0x34d57d25f3b9(60787, "Bz&S")
r4 = global_xF[(r5 + "lt")]
r5 = func_n_0x34d57d25f3b9(58819, "5C8Q")
r5 = (r5 + func_n_0x34d57d25f3b9(17159, "SldQ"))
return r4[(r5 + "t")]
```

A function called with a number and a string, its result concatenated with the next — a good guess that it performs string deobfuscation.

SAME FRAGMENT, DECODED

before – straight from View8 (comments mine)

```
r5 = func_n_0x34d57d25f3b9(60787, "Bz&S") // "defau"  
r4 = global_xF[(r5 + "lt")]  
r5 = func_n_0x34d57d25f3b9(58819, "5C8Q") // "globa"  
r5 = (r5 + func_n_0x34d57d25f3b9(17159, "SldQ")) //  
"lAgen"  
return r4[(r5 + "t")]
```

The filter resolves every
chunk back to plain text.

after – our filter



```
r4 = global_xF["default"]  
return r4["globalAgent"]
```

HOW THE STRINGS UNLOCK

chunk the strings → RC4 per-chunk key → Base64 → one big array

```
// the function that holds the chunked strings
function func_KV_0x18c3e8c9a1c1()
{
  Scope[10824][2] = new ["s8ohWR3dRx8", "ffddSSo6sW", ... thousands more ...]
}
```

```
// a call to the deobfuscating function
r2 = func_xt_0x274f42c4e909(71692, "%]hf")
                        ↳ RC4 key
                        ↳ index (to be transformed)
```

We know the algorithm & keys. The hard part is **finding the correct chunk**.

WHY THE INDEX IS HARD

```
// the call  
r2 = func_xt_0x274f42c4e909(71692, "%]hf")
```



```
// what's underneath the function  
function func_xt_0x274f42c4e909(a0, a1)  
{  
    r1 = (a0 + Scope[1][8]["K"])  
    return func_r_0x24543ecee91(a1, r1)  
}
```

The index doesn't point straight to the chunk — it goes through an **arithmetic transformation**.

```
index → +/- → +/- → +/- → real chunk chained decoders, each adds or subtracts
```

Some values aren't local constants — they come via scope. Repeat every operation, but populate the scopes first.

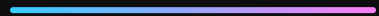
CRACKING THE PARENT

Each chain ends in one heavily obfuscated parent that adds a final constant.

Don't deobfuscate it — treat it as a **black box** and crack the value.

Only one such parent per file needs cracking.

BOUNDED SEARCH

0  array size

Pick a candidate → decrypt the chunk → is it plausible, printable text?

x3

Readable ≠ correct. Accept a value only if it works in at least three test cases.

STRINGS AT SCALE

~130k

encoded chunks (avg)

min 19k · max 217k

~10k

decoder configurations (avg)

min 2.2k · max 13k

~2 → ~1.5 min

average runtime, without vs with
CSV cache

	MINIMUM	MEDIAN	MAXIMUM
no	0.6	1.7	4.6
cache			
with	0.5	1.2	3.0
cache			

DATASET

github.com/hasherezade/jsceal_datasets/tree/main/logs/sessions_23_samples

PASS 1 · STRINGS

THE FIRST REAL ARTIFACTS

HIDDEN POWERSHELL EXECUTION

```
"powershell.exe"  
"-NoProfile -WindowStyle Hidden"  
"-ExecutionPolicy Bypass -Command"  
"Add-MpPreference -ExclusionPath"
```

BROWSER SECRETS, COOKIES & OAUTH

```
"iterInstalledBrowsers"  
"getCookies" "getPasswords"  
"--user-data-dir=" "--profile-directory="  
"oauth_token" "save0AuthToken"
```

consolidate chunks → substitute back →
hide decoders

+ a listing of every decoded string,
written as its own file

CRYPTOCURRENCY BALANCES

```
".phantom-labs.vault." "binance"  
"totalBalanceInUSDT"  
"free_margin_usd"  
"customer_account_USDT_balance_available"
```

THE ATTACKER'S EMBEDDED PUBLIC KEY

```
-----BEGIN PUBLIC KEY-----  
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAtRdWL/ucoH  
+ZnVuxHrx2cTbwEY2LucyUqEJVL6trmNYaJTFX9qDYA8Z4V0aF086M  
Hg0cY1mJ8NALzTqDt20C ...  
-----END PUBLIC KEY-----
```

UNFLATTENING

The most important functions are flattened: code split into numbered blocks, run by a **state machine**.

Their intended order is stored as a **string** — for example **"3|2|1|0|4"**.

THE DISPATCHER LOOP

```
r6["jGBGz"] = "3|2|1|0|4"
r1 = r6
r7 = r1["jGBGz"]
r6 = r7["split"]
r3 = r6("|")
while (true)
{
    r7 = Number(r4)
    r4 = (Number(r4) + 1)
    r6 = r3[r7]
    if (!r6 === "0") { [block0]; continue; }
    if (!r6 === "1") { [block1]; continue; }
    if (!r6 === "2") { [block2]; continue; }
    if (!r6 === "3") { [block3]; continue; }
    if (!r6 === "4") { [block4]; continue; }
}
```


PROPAGATING THE ORDER

The order string is unreadable until Pass 1 — then it must reach the flattened function.

It is decoded upstream, in the declarer, and passed **downstream** via scope into place.

RECOVERING THE ORDER • PASS 1

```
func_1:  
Scope[843][9]["K"] = func_xt_0x274f42c4e909(99278, "h^gm")  
  
func_2:  
r6["jGBGz"] = Scope[843][9]["K"]
```



```
func_1:  
Scope[843][9]["K"] = "3|2|1|0|4"  
  
func_2:  
r6["jGBGz"] = "3|2|1|0|4"
```

RESTORING THE ORDER

the dispatcher loop

```
while (true)
{
    r7 = Number(r4)
    r4 = (Number(r4) + 1)
    r6 = r3[r7]
    if (!r6 == "0") { [block0]; continue; }
    if (!r6 == "1") { [block1]; continue; }
    if (!r6 == "2") { [block2]; continue; }
    if (!r6 == "3") { [block3]; continue; }
    if (!r6 == "4") { [block4]; continue; }
}
```

RESTORING THE ORDER

mind the notation

```
while (true)
{
  r7 = Number(r4)
  r4 = (Number(r4) + 1)
  r6 = r3[r7]
  if (!r6  $\equiv$  "0") { [block0]; continue; }
  if (!r6  $\equiv$  "1") { [block1]; continue; }
  if (!r6  $\equiv$  "2") { [block2]; continue; }
  if (!r6  $\equiv$  "3") { [block3]; continue; }
  if (!r6  $\equiv$  "4") { [block4]; continue; }
}
```

CUSTOM NOTATION

$!r6 \equiv "0"$

means

$r6 \not\equiv "0"$

View8 pseudocode, not literal JS — the **!** negates the whole comparison.

RESTORING THE ORDER

dispatcher → linear block order

```
while (true)
{
    r7 = Number(r4)
    r4 = (Number(r4) + 1)
    r6 = r3[r7]
    if (!r6 == "0") { [block0]; continue; }
    if (!r6 == "1") { [block1]; continue; }
    if (!r6 == "2") { [block2]; continue; }
    if (!r6 == "3") { [block3]; continue; }
    if (!r6 == "4") { [block4]; continue; }
}
```

ORDER STRING · FROM PASS 1

`"3|2|1|0|4"`

run blocks 3 · 2 · 1 · 0 · 4

RESTORED · LINEAR ORDER

```
// "3|2|1|0|4"
[block3];
[block2];
[block1];
[block0];
[block4];
```



REBUILDING THE LOGIC

FLATTENED · MULTIPLE CONTINUES

```
while (true) // the dispatcher loop
{
    r15 = Number(r4)
    r4 = (Number(r4) + 1)
    r14 = r3[r15]
    if (!r14 == "0")
    {
        // other chunks... [...]
    }
    CHUNK 0 -----
    // Chunk 0:
    r15 = r2["Uugef"]
    if (r15(r11, r12))
    {
        ACCU = 0
        continue // not the end of the chunk...
    }
    r15 = r2["PgJCU"]
    r17 = r2["LqFvW"]
    r17 = r17(r11, r12)
    if (r15(r17, r5))
    {
        ACCU = 1
        continue // not the end of the chunk...
    }
    return -1
}
break
```



REBUILT · NESTED IF / ELSE

```
// Chunk 0:
r15 = r2["Uugef"]
if (r15(r11, r12))
{
    ACCU = 0
}
else // added else statement
{
    r15 = r2["PgJCU"]
    r17 = r2["LqFvW"]
    r17 = r17(r11, r12)
    if (r15(r17, r5))
    {
        ACCU = 1
    }
    else // added else statement
    {
        return -1
    }
}
```

one block, multiple conditional
continues → rebuild as nested if /
else

THE FUNCTION, LINEAR AGAIN

real func_Mz – nested dispatch
peeled into a straight line

FLATTENED · NESTED DISPATCH

```
if (!r6 == "1")
{
    if (!r6 == "2")
    {
        if (!r6 == "3")
        {
            // ...
        }
        if (r1["ykkYm"](a0, "ws"))
        {
            return global_Nb["default"]["globalAgent"]
        }
        continue
    }
    if (r1["SeAyf"](a0, r1["yHrsY"]))
    {
        return global_Tb["default"]["globalAgent"]
    }
    continue
    a0 = a0["split"](":")[0]
}
```

LINEAR AGAIN:

```
// ...
if (r1["SeAyf"](a0, r1["yHrsY"]))
{
    return global_Tb["default"]["globalAgent"]
}
if (r1["ykkYm"](a0, "ws"))
{
    return global_Nb["default"]["globalAgent"]
}
// ...
```

After unflattening the function starts to reveal its [shape](#).

PROXIES

a proxy – calls indirectly

Its only role is to forward a call.
Proxies chain into rabbit holes that
bury the real call tree.

Filter: mark → resolve target → replace
→ hide.

COLLAPSING A PROXY CALL

```
- function func_INBzN_0x16abfdb5cc69(a0, a1, a2, a3)
- {
-   return a0(a1, a2, a3)
- }

@@ -22213,7 +21565,7 @@ function func_J_0x16abfdb59891()
-   }
-   else
-   {
-     ACCU = func_INBzN_0x16abfdb5cc69(func_k_0x16abfdb5a2d9, <this>, null, null)
+     ACCU = func_k_0x16abfdb5a2d9(<this>, null, null)
-   }
- }
```

OPERATION WRAPPERS

even a single operator becomes a function

A subtraction, a division, an `in` check — each hidden behind its own function, repeated under many names and layered.

~18,500

wrapper functions hidden in one run — noise, not features

FOUR WRAPPERS, ONE OPERATION EACH

```
function func_wcmWN_0x35459f2fab89(a0, a1)
{
    return a0 in a1
}
function func_eBvDY_0x35459f2fa789(a0, a1)
{
    return (a0 - a1)
}
function func_wNPYv_0x35459f2fa689(a0, a1)
{
    return (a0 / a1)
}
function func_oEEDc_0x35459f2faa89(a0, a1)
{
    return a0(a1)
}
```


PEELING THE LAYERS

One line, filled with its literal meaning — layer by layer.

```
function func_value_0x24149a8e3d19(a0)
{
  [...]
  r10 = Scope[846][21][r5(Scope[846][3]["N", Scope[846][3]["M"]])
  r13 = r5(Scope[846][3]["k", Scope[846][3]["Q"])
  r12 = r0[(r13 + "h")]
  r10 = r10(r12, a0)
```

01 Initial decompiled code

1 / 6

PEELING THE LAYERS

One line, filled with its literal meaning — layer by layer.

```
function func_value_0x24149a8e3d19(a0)
{
  [...]
  r10 = Scope[846][21][func_me_0x24149a8e4421(12568, "%]hf")] // "qQNNx"
  r13 = func_me_0x24149a8e4421(34408, "[Jy3") // "lengt"
  r12 = r0[(r13 + "h")]
  r10 = r10(r12, a0)
```

02 Resolve the string-deobfuscation calls

2 / 6

PEELING THE LAYERS

One line, filled with its literal meaning — layer by layer.

```
function func_value_0x24149a8e3d19(a0)
{
  [...]
  r10 = Scope[846][21]["qQNNx"] // func_qQNNx_0x24149a8dfe99
  r12 = r0["length"]
  r10 = r10(r12, a0)
```

03 A revealed string is a dictionary key → a function

3 / 6

PEELING THE LAYERS

One line, filled with its literal meaning — layer by layer.

```
function func_value_0x24149a8e3d19(a0)
{
  [...]
  r12 = r0["length"]
  r10 = func_qQNNx_0x24149a8dfe99(r12, a0) // → func_PQxQy_0x24149a8dd581
```

O4 Inline the resolved function at its call site

4 / 6

PEELING THE LAYERS

One line, filled with its literal meaning — layer by layer.

```
function func_value_0x24149a8e3d19(a0)
{
  [...]
  r12 = r0["length"]
  r10 = func_PQxQy_0x24149a8dd581(r12, a0)
```

05 Substitute the proxy with its real target

5 / 6

PEELING THE LAYERS

One line, filled with its literal meaning — layer by layer.

```
function func_value_0x24149a8e3d19(a0)
{
  [...]
  r12 = r0["length"]
  r10 = (r12 - a0)
```

06 The wrapper collapses into one atomic operation

6 / 6

PEELING THE LAYERS

each pass feeds the next

a recovered string

↓ reveals

a field name → a dictionary entry

↓ points to

a proxy → a wrapper

↓ becomes

one operation

before – proxied wrappers

```
// ...  
if (r1["SeAyf"](a0, r1["yHrsY"])) { return global_Tb["default"]["globalAgent"] }  
if (r1["ykkYm"](a0, "ws")) { return global_Nb["default"]["globalAgent"] }  
// ...
```

↓ resolved

```
// ...  
if (a0 === "https") { return global_Tb["default"]["globalAgent"] }  
if (a0 === "ws") { return global_Nb["default"]["globalAgent"] }  
// ...
```

ykkYm, SeAyf were comparison wrappers; yHrsY held "https".



Scan for the full
before & after of the
running function

SCAN FOR THE FULL BEFORE & AFTER OF THE RUNNING FUNCTION



gist.github.com/hasherezade/c14985f37f8512f2c3c7b7cf8807541e

MILESTONE

THE CLEANED OUTPUT

- ✓ strings are present
- ✓ important flows linear again
- ✓ many proxies & wrappers gone
- ✓ decoders & helpers hidden
- ✓ splittable into logical branches

ONE PROBLEM REMAINS

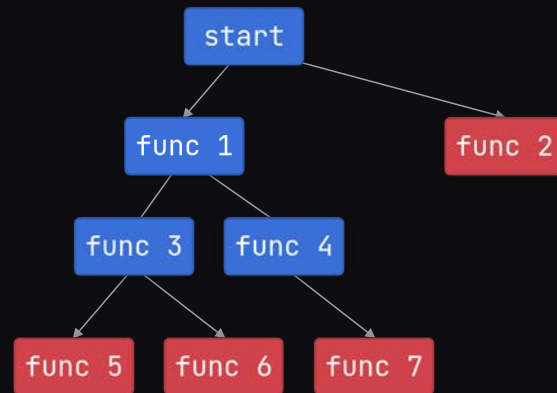
Names are **gibberish** — and unrecoverable.

Manual renaming is too difficult at scale.

LLM-ASSISTED NAMING

- 1 Build a dependency graph from the entry point
- 2 Modes: basic (direct calls) vs greedy (every reference)
- 3 Leaf-first — fewest unresolved deps first
- 4 Propagate each new name into remaining bodies

LEAF-FIRST



Red leaves named first → propagate up to start

BULK CSV MAPPING

Bulk mode returns a CSV mapping each **old identifier** to a **proposed name**.

The same file doubles as a **resumable cache** — an interrupted run picks up where it stopped.

RENAME_MAP.CSV

```
func_FL_0x39fbdd9eda9,func_callSuper_0x39fbdd9eda9
func_unknown_0x26b1a1babb39,func_escapeChar_0x26b1a1babb39
func_unknown_0x20b39019e6d1,func_setValue_0x20b39019e6d1
func_unknown_0xc4a9532d969,func_emitResponseData_0xc4a9532d969
func_de_0x3615a23b6001,func_bitwiseAnd_0x3615a23b6001
func_unknown_0x16d39d6a9309,func_isFulfilled_0x16d39d6a9309
func__write_0x37aee3f919d9,func_write_0x37aee3f919d9
func_ee_0x95e5d24cb31,func_flushStreams_0x95e5d24cb31
func_unknown_0x32b5d60b0cf1,func_unregister_0x32b5d60b0cf1
func_unknown_0x30be66b9bf99,func_createEncoderStream_0x30be66b9bf99
func_unknown_0x1c3c9cc0ec01,func_matchesAttribute_0x1c3c9cc0ec01
func_unknown_0x31a18b0ddf19,func_parseJsonResponse_0x31a18b0ddf19
func_zu_0x3cc839129751,func_cloneBody_0x3cc839129751
func__err_0x119b51f94589,func_onError_0x119b51f94589
func_m_0xbd9c56cc7a9,func_HttpParserError_0xbd9c56cc7a9
func_Pe_0x15e5e5ffbc01,func_addSchemas_0x15e5e5ffbc01
func_unknown_0x141d9a386581,func_readPayloadLength_0x141d9a386581
func_unknown_0x3615a2389cb9,func_decryptBlock_0x3615a2389cb9
```

THE RUNNING EXAMPLE — REAL OUTPUT

getGlobalAgent()

REAL DEOBFUSCATED OUTPUT

```
function func_Mz_0x10000000f(a0)
{
  r7 = a0["split"]
  r7 = r7(":")
  a0 = r7[0]
  if (a0 === "http")
  {
    return global_Nb["default"]["globalAgent"]
  }
  if (a0 === "https")
  {
    return global_Tb["default"]["globalAgent"]
  }
  if (a0 === "ws")
  {
    return global_Nb["default"]["globalAgent"]
  }
  if (a0 === "wss")
  {
    return global_Tb["default"]["globalAgent"]
  }
  r8 = "Invalid protocol"
  ACCU = Error(r8)
  return undefined
}
```

decoder calls → flattened → linear → [named](#)

LLM'S GUESS AT THE ORIGINAL

```
function getGlobalAgent(url) {
  const protocol = url.split(":")[0];
  if (protocol === "http") {
    return http.default.globalAgent;
  }
  if (protocol === "https") {
    return https.default.globalAgent;
  }
  if (protocol === "ws") {
    return http.default.globalAgent;
  }
  if (protocol === "wss") {
    return https.default.globalAgent;
  }
  throw new Error("Invalid protocol");
}
```

Left: the real decompiled, deobfuscated body — register slots and global references intact. Right: the LLM's reconstruction of how the original source likely read.

IS THE NAME TRUE?

WHAT THE FILTER CAN REJECT

empty

no answer at all

too short

a letter or two

malformed

not a valid identifier

unknown

the model gave up

– this protects the format, not the truth

WHETHER A NAME IS **TRUE** HAS TO BE TESTED SEPARATELY

SONNET VS GPT

Claude Sonnet 4.6 vs GPT-5.4-mini

The same cleaned payload through both models.

These were the models I used while conducting the research — not perfectly matched vendor tiers, but practical configurations for processing a payload this large.

DATASET FOR THE EXPERIMENT

github.com/hasherezade/jsceal_datasets/tree/main/e27ae/ai_labels/session1

21,154

functions named by both models

9.3%

chose the exact same name (1,971)

Exact wording ≠ meaning. Different names may denote the same functionality.

JUDGING IN CONTEXT

renamed pickle → View8 →

function subtree from a root

Output splitting makes portions small enough to read — or to hand back to an LLM for another round.

Started at the entry point, followed the branches that set up the malware logic.

SEMANTIC SENSE ACROSS 142 HIGH-CONTEXT ROOTS

117



both
sensible

22



Sonnet
only

3



GPT only

Accurate & informative: [Sonnet 128/142](#) · GPT 30 (+90 right general concept). The gap was mainly precision.

A NAME IS A HYPOTHESIS

GPT bootKey

Sonnet
initBootKeyModule

Both useful — Sonnet says what it does (inits a module, exports getBootKey).

GPT findCertificate

Sonnet
removeCertificate

Not cosmetic — the body deletes/splices an entry from the cert store.

Sonnet
decryptLocalStateFile

Confident & **wrong** — it actually parses a DPAPI master-key file (Protect dir), decrypts, verifies HMAC.

The model helps us **find and navigate** the logic. The code remains the **evidence**.

THE PAYOFF – ALL FROM DEOBFUSCATED CODE

WHAT WE RECOVERED

one selected payload · MD5 `e27ae65977287bdfb7b0e15fd3603f85`

WHAT WE RECOVERED

CRYPTOCURRENCY THEFT, AT SCALE

50+

platform-specific handlers

30+

pure crypto exchanges

Binance

Bybit

OKX

Kraken

+ many more

Names surfaced the handler cluster; targets verified through hostnames, routes, response fields, and the data each body saves. Payment & P2P platforms in there too.

WHAT WE RECOVERED

WALLETS & EXTENSIONS

10

wallet-extension
initializers + a
hardware-wallet
path

MetaMask

Phantom

Rabby

Trust Wallet

+ more

Evidence is unusually direct — the bodies carry extension types, extension IDs, popup paths, and patch configuration.

WHAT WE RECOVERED

BROWSER & CREDENTIAL THEFT

100+

cookie-related functions – bundled parsers plus
JSCeal-specific read/extract paths,
including Facebook and OAuth

Windows creds — DPAPI structures + master-key files, LSA registry, derive keys, decrypt

Windows Hello + passkey collection

Crypto seed phrases

Puppeteer + stealth plugins — replay stolen cookies, automate Google login challenges, grab Android OAuth tokens

WHAT WE RECOVERED

TELEGRAM - KEYLOG - SCREENSHOTS

TELEGRAM

Find Telegram Desktop → enter `tdata`
→ read session files → hand to storage
handlers

KEYLOGGING

Start/stop a keyboard-capture
component; a keydown subscription —
not a Windows hook

SCREENSHOTS

Screen capture + window enumeration
and control

Buried before deobfuscation. Verifiable modules after.

WHAT WE RECOVERED — THE ONE I SLOW DOWN FOR

THE LOCAL MITM PROXY

leg 1
proxy setup

leg 2
certificate generation

leg 3
certificate installation

generation stack: `generateKeyPair` → `createCertificate (subject + issuer)` → `signCertificate`

```
certutil -addstore -f root <certificate path>
```

Installs an **attacker-controlled certificate** into the trusted root store — now the local proxy can intercept HTTPS.

STATIC INTENT, MEET RUNTIME PROOF

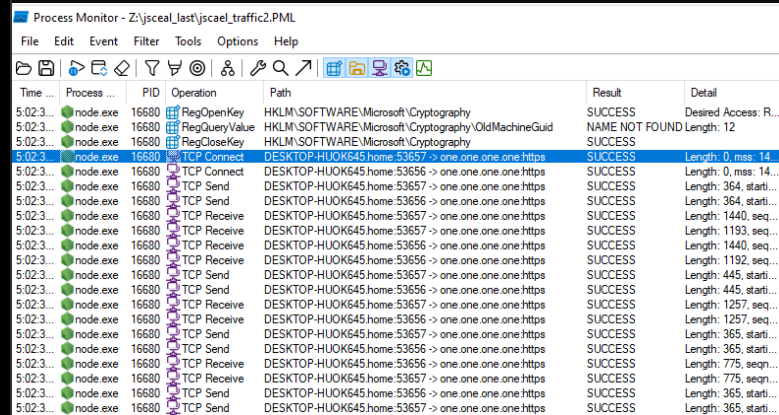
THE RECOVERED ENTRY POINT

module entry – malware init, after deobfuscation

```
ACCU = func_copyProperties_0x93e23cefe71(global_QU, r3)
global_s7e = {}
global_G7e = {}
ACCU = func_requireCluster_0x93e23cf3479()
r1 = (require("dns"))["setServers"]
r3 = new [0, 0]
r3[0] = "1.1.1.1"
r3[1] = "8.8.8.8"
ACCU = r1(r3)
ACCU = func_setupWorker_0x3fa27d771a51(__filename)
if (func_setupWorker_0x3fa27d771a51(__filename))
{
    ACCU = func_initializeApplication_0x217bb6195779()
    ACCU = func_exportModule_0x93e23cf1a31(global_s7e)
}
else
{
    ACCU = func_initializeApplication_0x7b2a97682c9()
    ACCU = func_exportModule_0x93e23cf1a31(global_G7e)
}
r0 = ACCU
return ACCU
```

A run shows one query — the recovered code shows the whole entry point that produced it. Static code shows the intent.

Process Monitor – the DNS query it triggers



The screenshot shows the Process Monitor application window with a filter applied to show only network events. The event log displays a series of TCP connections and data exchanges between the local machine (DESKTOP-HUOK645) and a remote host (one.one.one.one). The selected event is a TCP Connect operation, which corresponds to the DNS query mentioned in the text.

Time ...	Process ...	PID	Operation	Path	Result	Detail
5:02:3...	node.exe	16680	RegOpenKey	HKLM\SOFTWARE\Microsoft\Cryptography	SUCCESS	
5:02:3...	node.exe	16680	RegQueryValue	HKLM\SOFTWARE\Microsoft\Cryptography\OldMachineGuid	NAME NOT FOUND	Length: 12
5:02:3...	node.exe	16680	RegCloseKey	HKLM\SOFTWARE\Microsoft\Cryptography	SUCCESS	
5:02:3...	node.exe	16680	TCP Connect	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 0, mss: 14...
5:02:3...	node.exe	16680	TCP Connect	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 0, mss: 14...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 364, starti...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 364, starti...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 1440, seq...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 1193, seq...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 1440, seq...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 1192, seq...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 445, starti...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 445, starti...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 1257, seq...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 1257, seq...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 365, starti...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 365, starti...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 775, seq...
5:02:3...	node.exe	16680	TCP Receive	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 775, seq...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53656 -> one.one.one.one:https	SUCCESS	Length: 365, starti...
5:02:3...	node.exe	16680	TCP Send	DESKTOP-HUOK645:home:53657 -> one.one.one.one:https	SUCCESS	Length: 365, starti...

The runtime trace confirms behavior.

BEYOND ONE SAMPLE

TESTED ACROSS 23 SAMPLES

23

unique payloads · several months

23 / 23 FINISHED WITH ANALYZABLE OUTPUT

232k

string-decoder
configurations resolved

1.78M

decoded string entries
exported

479k

final cleanup rewrites

4.6min

avg per sample, no cache
shortest 1.4 · longest 7.6

Strings, flattened functions, capability clusters — all recovered unattended.

DATASET

github.com/hasherezade/jsceal_datasets/tree/main/logs/sessions_23_samples

HONEST ABOUT IT

LIMITS

V8 disassembly is version-sensitive — a different Node version will need a matching disassembler

Filters are pattern-based — may need tuning for other settings or output

Native .node modules remain outside the pipeline

LLM names are navigation aids, not proof

Not perfect source recovery — but a [practical path](#) to code you can read, validate, and compare.

DATED BY DESIGN

A MOVING TARGET

WHAT HAS MOVED

Context windows and storage expanded dramatically over the year of this work — some constraints here are already less severe.

The model comparison was a snapshot, not a ranking.

WHAT DOESN'T MOVE

The layers still come off by computation, not inference — a model with a code interpreter can decrypt the chunks, but then it is implementing its own version of the analogous pass.

Nearly 3 million encoded chunks across 23 samples, decoded the same way every time — inspectable and verifiable.

The passes give us **control over the transformation and over the output**. The model-specific parts are dated by design — that control is the durable part.

TAKE IT WITH YOU

RELEASED TOOLKIT

OPEN-SOURCE

- > View8 integration
- > output splitting
- > deobfuscation filters
- > dependency-aware LLM renamer

github.com/hasherezade/jsc_deobfuscator



SCAN TO ACCESS

[github.com/hasherezade/
jsc_deobfuscator](https://github.com/hasherezade/jsc_deobfuscator)

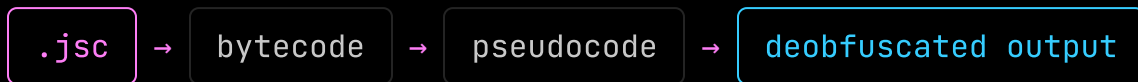
CREDIT

View8 is a separate project by Moshe Marelus — I contributed improvements and hope it keeps growing.

github.com/suleram/View8

CONCLUSIONS

FROM SEALED BOX TO READABLE CODE



Compiled V8 bytecode doesn't have to be a **sealed box**.

- Crossed the gap with a matching disassembler, View8, dedicated passes, and LLM-assisted navigation.
- Not the original JavaScript — but sufficient for practical threat analysis, and for tracking the family as it evolves.



THANK YOU

Aleksandra “Hasherezade” Doniec

Check Point Research

hasherezade.net speakerdeck.com/hshrzd



tools & slides

.JSC → BYTECODE → PSEUDOCODE → DEOBFUSCATED OUTPUT →
ANALYZE IT LIKE CODE AGAIN



SAMPLE SET - 23 PAYLOADS

MD5 · JSceal .jsc payloads
analyzed 2026

01 03f4e47b9c2283c32bb8f8f042ce6e41
02 0b8015cbb1ffdc6efe6a306ff5b1115f
03 1026743185dfa10e9ddc21b5a4c578d5
04 201f28b5e62e52e269757930f941c774
05 2fe27eb8c99626e8c02e4bfd02aca962
06 376ec4dbc3363fa7131367e4c6327a46
07 462195f7f8033df7371e899fe9bc51de
08 499184635d56a9827d2059256a35e530
09 533d0b93ea03cd5bab4eec0f0ebadd03
10 68ac84a8470d1f365f0bb2f37b6256d5
11 6e023b9b3097a2dba311cb06a91fe259
12 7b659fa5c93af29c4e11d8c8be437058

13 8fb3e6acb2024601eba0ba484091ff3d
14 af105a6d4dc10b2bfefd75e917245523
15 b2dad3f88b7f6870f83eb1ad852b7f7e
16 d064dfaaef30c057b832c79996c35e89
17 d5b4137135cf121e3ea07b1c81fe1108
18 e26687982d924ffebe6fbf2d9d43350
19 e27ae65977287bdfb7b0e15fd3603f85
20 e711a90b5ece5380e1acaed56827e8d5
21 e81b35b76b4d97751c0724bc0c7f3b83
22 e8b5448b4f7b013e8c6191b20d3f8291
23 fd4494c555adda2eb54b88f5c9c08801